

## Project: Custom VPC Setup with Bastion Host and NAT Gateway

**Services:** VPC, EC2, NAT Gateway, Bastion Host, Security Groups.

**Goal:** Build a secure private/public subnet architecture with internet access for private instances.

**Skills:** Routing tables, subnetting, SSH tunneling.

**Definition:** Custom VPC Setup with Bastion Host and NAT Gateway is a project that creates a secure AWS Virtual Private Cloud (VPC) with public and private subnets, where private EC2 instances access the internet through a NAT Gateway and can be securely managed via a Bastion Host, demonstrating controlled networking, routing, and access management.

### Scope of the Project:

- Design and implement a secure AWS network environment.
- Create a custom VPC with public and private subnets.
- Deploy EC2 instances in the private subnets.
- Configure a Bastion Host for secure SSH access to private instances.
- Set up a NAT Gateway to provide internet access for private instances.
- Manage routing tables to control traffic between subnets and the internet.
- Configure security groups to enforce network isolation and access control.

### Methodology:

#### 1.AWS Management Console (Manual Configuration Method)

- Uses core AWS services: VPC, EC2, NAT Gateway, Bastion Host, Route Tables, Security Groups.
- Provides clear understanding of public and private subnet architecture.
- Enables secure SSH access to private instances via Bastion Host.
- Allows outbound internet access for private instances using NAT Gateway.
- More cost-effective for academic and short-term projects, as resources can be manually created, tested, and deleted easily.

#### 2.AWS CloudFormation (AWS-Native Automated Method)

- Uses CloudFormation templates to automate infrastructure creation.
- Ensures consistent and repeatable deployment of AWS resources.
- Reduces manual configuration errors.
- Suitable for production and large-scale environments.
- Less cost-effective for small projects, as automation is better suited for repeated or long-term deployments.

### AWS Services Used in the Project:

#### 1.Amazon VPC (Virtual Private Cloud)

- **Definition:** A logically isolated virtual network in AWS where resources can be launched.
- **Purpose:** To provide complete control over networking, IP addressing, subnets, and routing.
- **Role in Project:** Forms the secure network environment separating public and private subnets for Bastion Host, NAT Gateway, and private EC2 instances.

## 2.Subnets

- **Definition:** Segments of a VPC that divide its IP address range into smaller networks.
- **Purpose:** To organize resources and apply different security and routing rules.
- **Role in Project:**
  1. **Public Subnet:** Hosts Bastion Host and NAT Gateway with internet access.
  2. **Private Subnet:** Hosts private EC2 instances with no direct internet access.

## 3.Internet Gateway (IGW)

- **Definition:** A horizontally scaled, redundant AWS component that allows communication between a VPC and the internet.
- **Purpose:** To provide internet connectivity for resources in public subnets.
- **Role in Project:** Enables Bastion Host and NAT Gateway to access the internet.

## 4.NAT Gateway

- **Definition:** A managed AWS service that enables instances in private subnets to connect to the internet without exposing them publicly.
- **Purpose:** To allow outbound internet access for private resources securely.
- **Role in Project:** Enables private EC2 instances to download updates or access external services while staying hidden from the internet.

## 5.Route Tables

- **Definition:** A set of rules, called routes, that determine where network traffic is directed.
- **Purpose:** To control traffic flow between subnets and the internet.
- **Role in Project:**
  1. **Public Route Table:** Routes internet-bound traffic from the public subnet to the Internet Gateway.
  2. **Private Route Table:** Routes internet-bound traffic from the private subnet to the NAT Gateway.

## 6.EC2 (Elastic Compute Cloud)

- **Definition:** Virtual servers that run applications in the cloud
- **Purpose:** To provide scalable compute capacity in AWS.
- **Role in Project:**
  1. **Bastion Host:** Provides secure SSH access to private EC2 instances.
  2. **Private EC2:** Hosts applications/services within the private subnet.

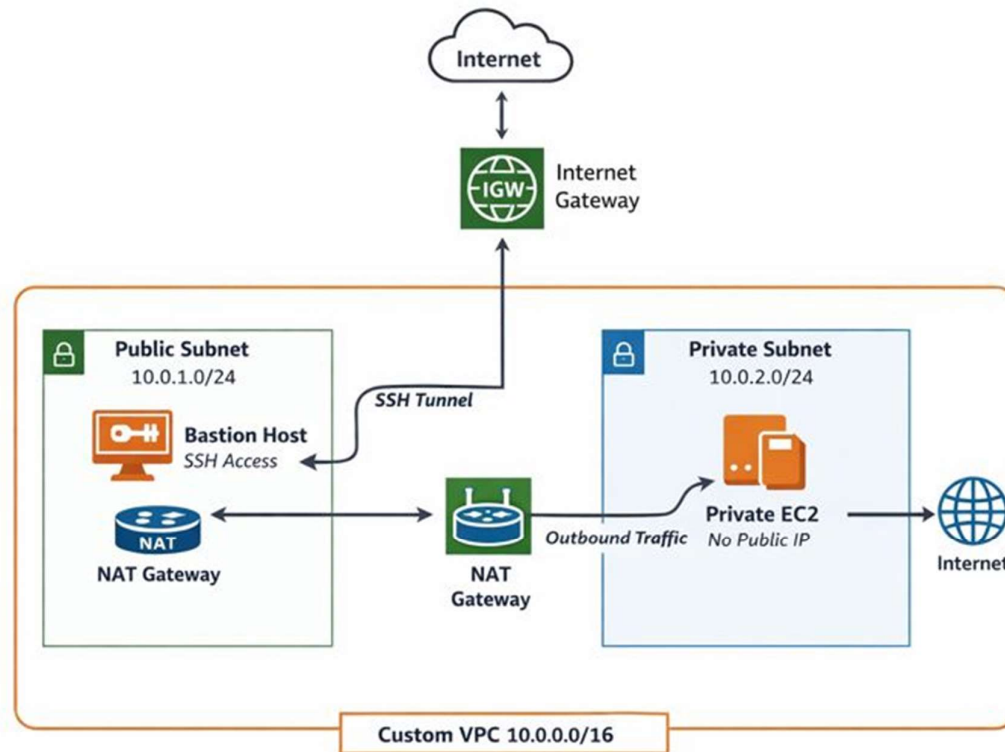
## 7.Security Groups

- **Definition:** Virtual firewalls that control inbound and outbound traffic for EC2 instances.
- **Purpose:** To secure resources by controlling network access.
- **Role in Project:**
  1. **Bastion Host SG:** Allows SSH only from user's IP.
  2. **Private EC2 SG:** Allows SSH only from Bastion Host and outbound traffic to NAT Gateway.

## 8.Elastic IP (EIP)

- **Definition:** A static, public IPv4 address for dynamic cloud resources.
- **Purpose:** Provides a consistent public IP for resources.
- **Role in Project:** Assigned to NAT Gateway to maintain a fixed outbound IP for private EC2 instances.

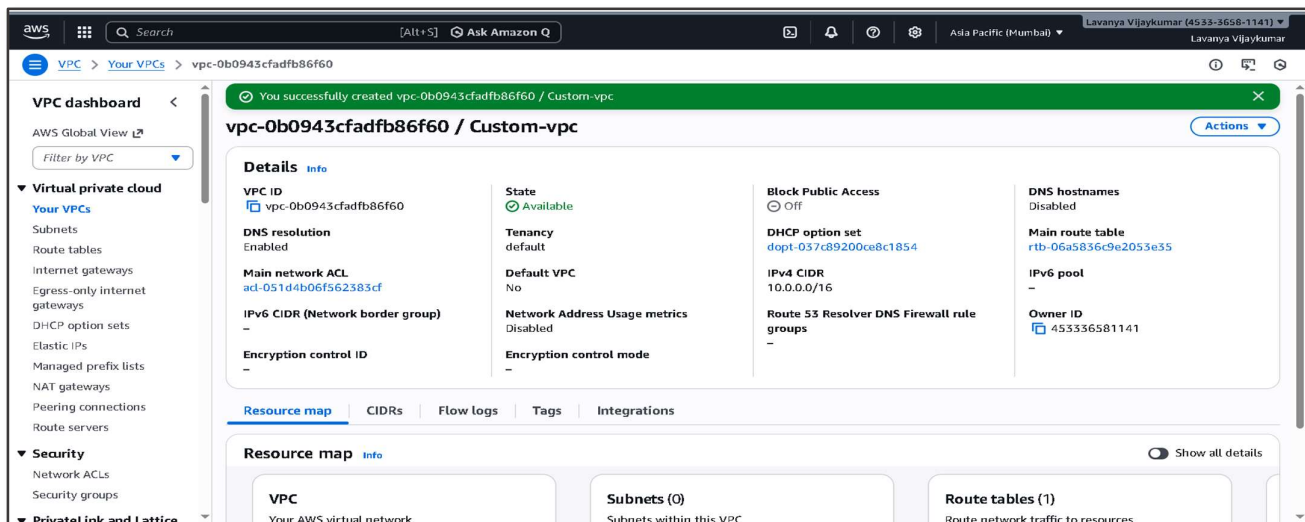
## Architecture Diagram:



## Implementation Steps to Do This Project:

### Step 1: Create a Custom VPC

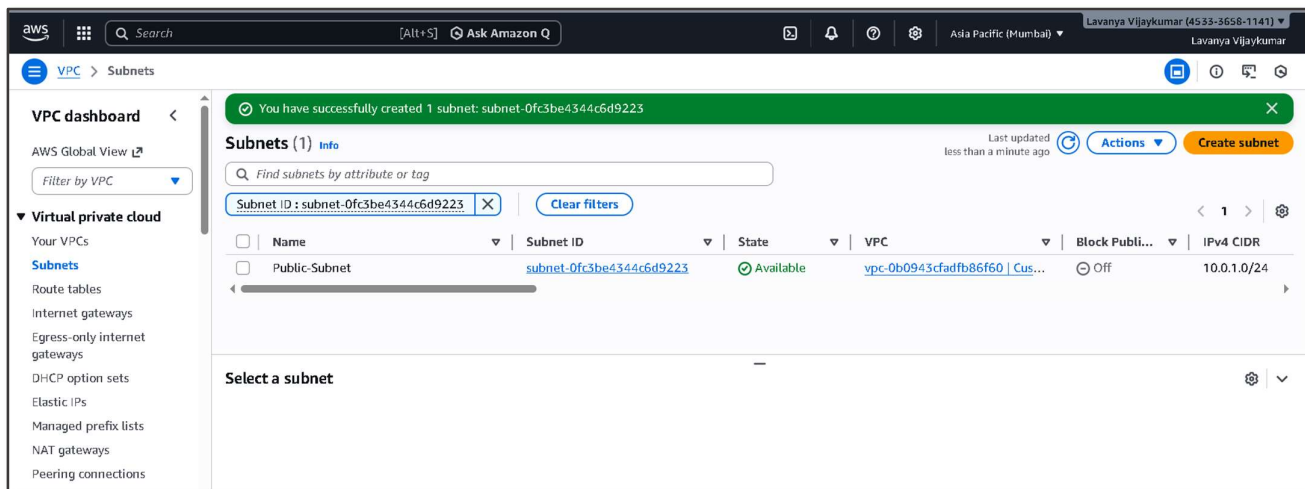
- Go to VPC → Your VPCs → Create VPC
- Enter:
  1. Name: Custom-VPC
  2. IPv4 CIDR: 10.0.0.0/16
- Click Create VPC.



## Step 2: Create Subnets

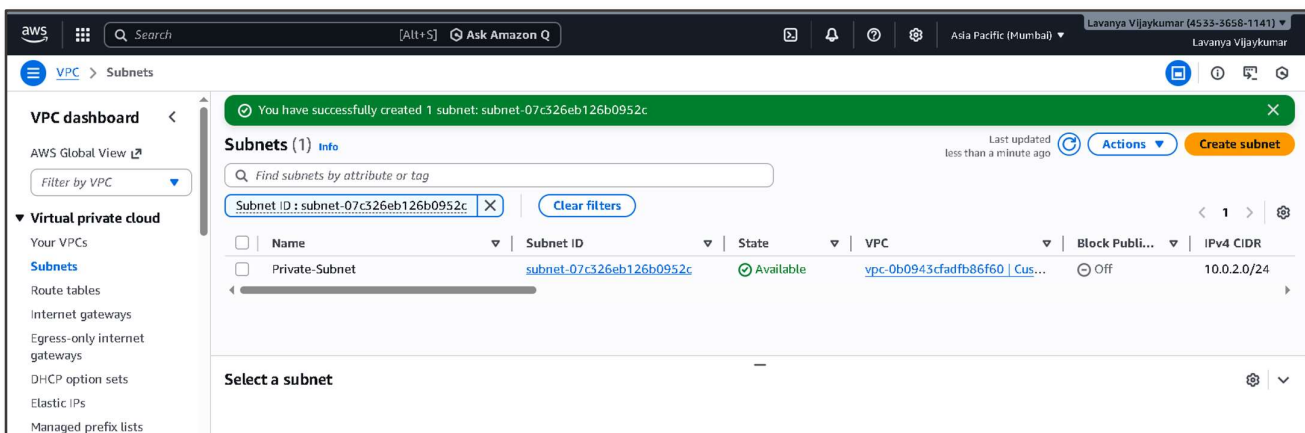
### 1.Public Subnet

- VPC → Subnets → Create subnet
- Select Custom-VPC
- Enter:
  1. Name: Public-Subnet
  2. CIDR: 10.0.1.0/24
- Click Create subnet



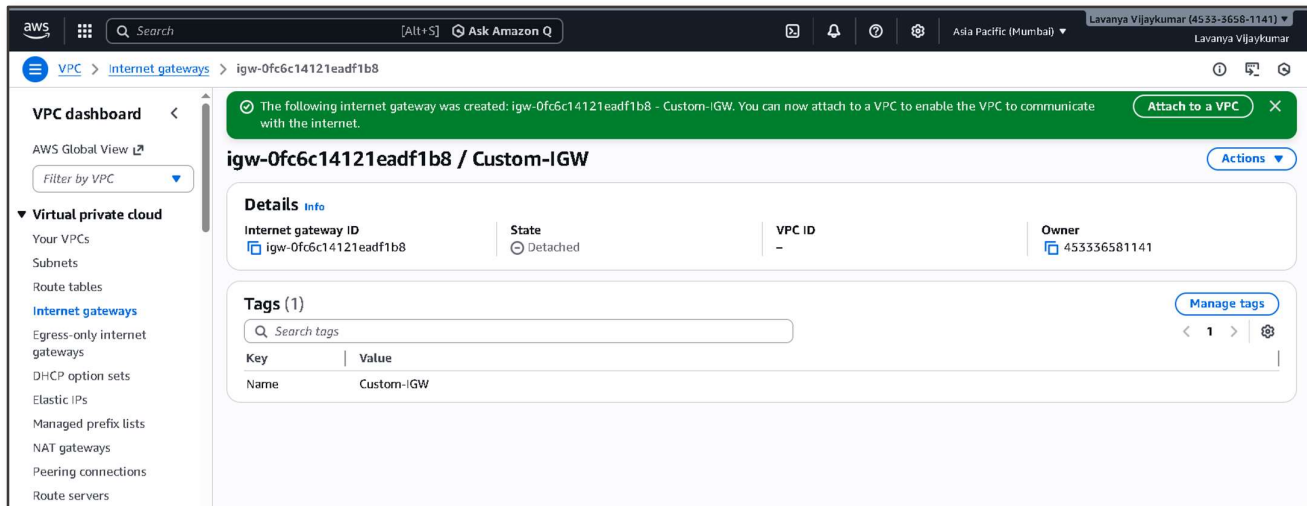
### 2.Private Subnet

- VPC → Subnets → Create subnet
- Select Custom-VPC
- Enter:
  3. Name: Private-Subnet
  4. CIDR: 10.0.2.0/24
- Click Create subnet

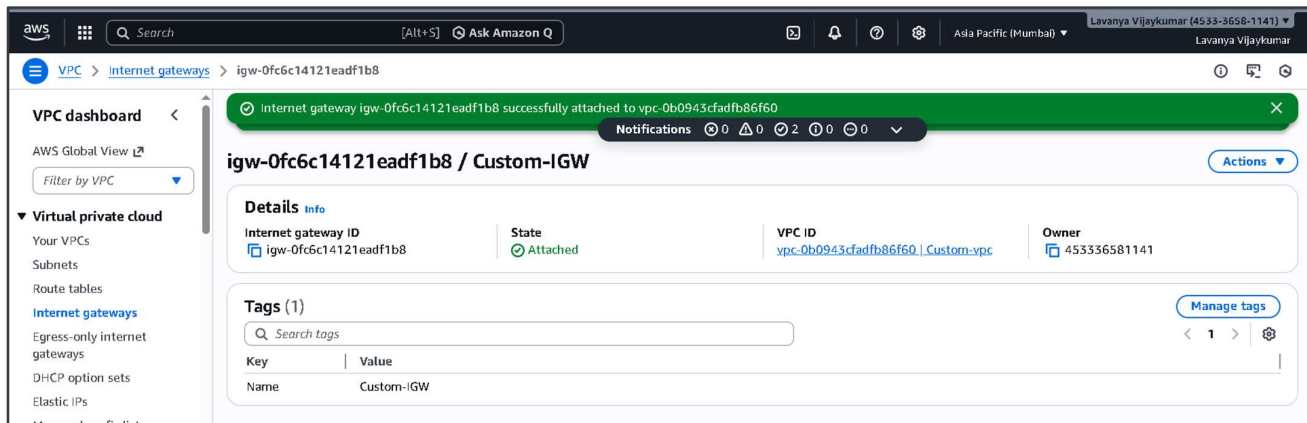


### Step 3: Create Internet Gateway and Attach to VPC

- Go to VPC → Internet Gateways → Create Internet Gateway.
- Enter Name: Custom-IGW.
- Click Create Internet Gateway.



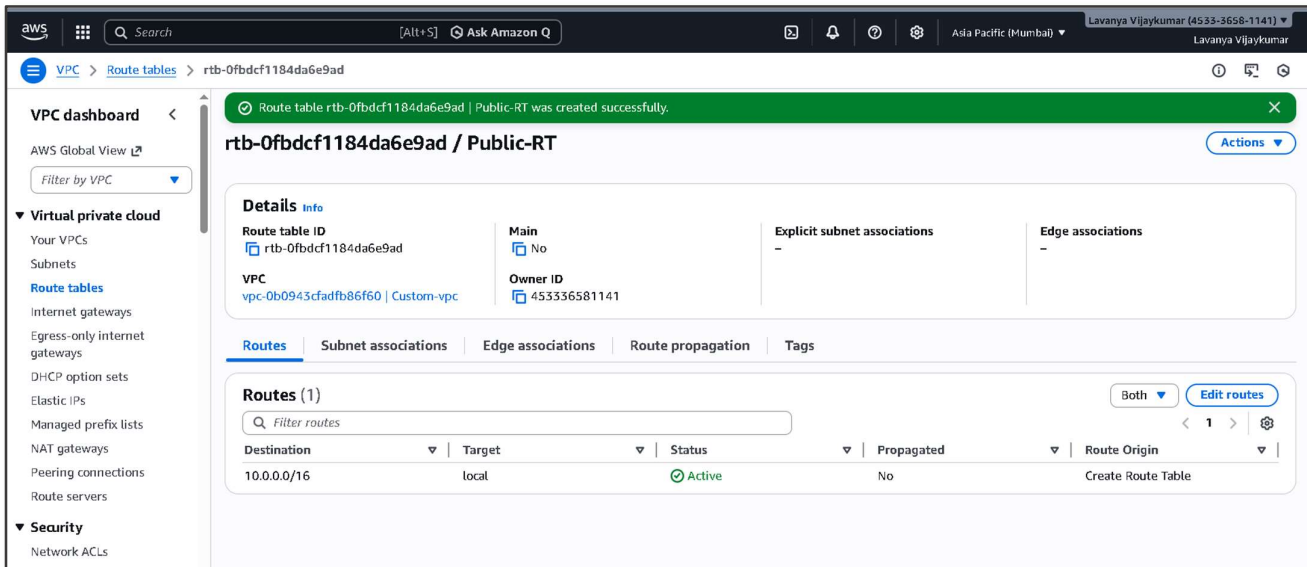
- After creation, select the newly created IGW (Custom-IGW) from the list.
- Click Actions → Attach to VPC.
- In the pop-up, select your Custom VPC (Custom-VPC) from the dropdown.
- Click Attach Internet Gateway.



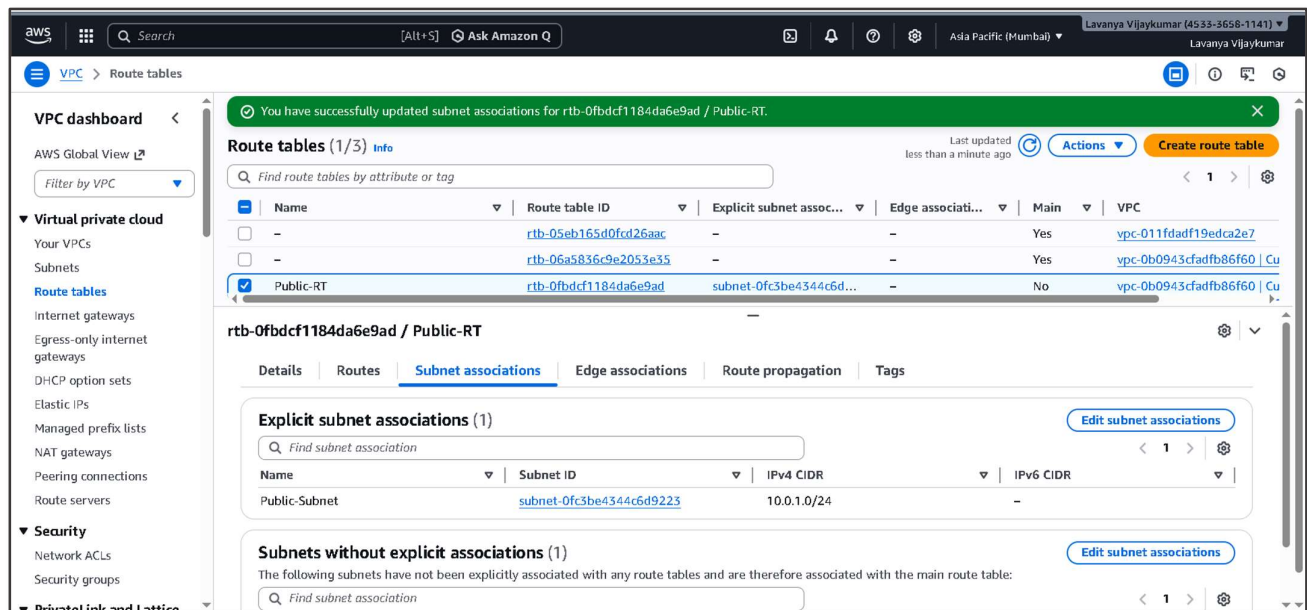
### Step 4: Create Route Tables

#### Public Route Table

- VPC → Route Tables → Create Route Table
- Name: Public-RT
- Select Custom VPC



- Select the Public-RT route table from the list.
- Go to the Subnet Associations tab.
- Click Edit Subnet Associations.
- Select the Public-Subnet
- Click Save.



- Go to the Routes tab → Click Edit routes → Add route:
- Destination: 0.0.0.0/0
- Target: Custom-IGW (Internet Gateway)
- Click Save routes.

**Updated routes for rtb-0fbdcf1184da6e9ad / Public-RT successfully**

**rtb-0fbdcf1184da6e9ad / Public-RT**

**Details**

Route table ID: [rtb-0fbdcf1184da6e9ad](#)

VPC: [vpc-0b0943cfadfb86f60](#) | Custom-vpc

Main: [No](#)

Owner ID: [453336581141](#)

Explicit subnet associations: [subnet-0fc3be4344c6d9223](#) / Public-Subnet

Edge associations: -

**Routes (2)**

| Destination | Target                                | Status | Propagated | Route Origin       |
|-------------|---------------------------------------|--------|------------|--------------------|
| 0.0.0.0/0   | <a href="#">igw-0fc6c14121eadf1b8</a> | Active | No         | Create Route       |
| 10.0.0.0/16 | local                                 | Active | No         | Create Route Table |

## Private Route Table

- VPC → Route Tables → Create Route Table
- Name: Private-RT
- Select Custom VPC

**Route table rtb-08b48e754f36380d8 | Private-RT was created successfully.**

**rtb-08b48e754f36380d8 / Private-RT**

**Details**

Route table ID: [rtb-08b48e754f36380d8](#)

VPC: [vpc-0b0943cfadfb86f60](#) | Custom-vpc

Main: [No](#)

Owner ID: [453336581141](#)

Explicit subnet associations: -

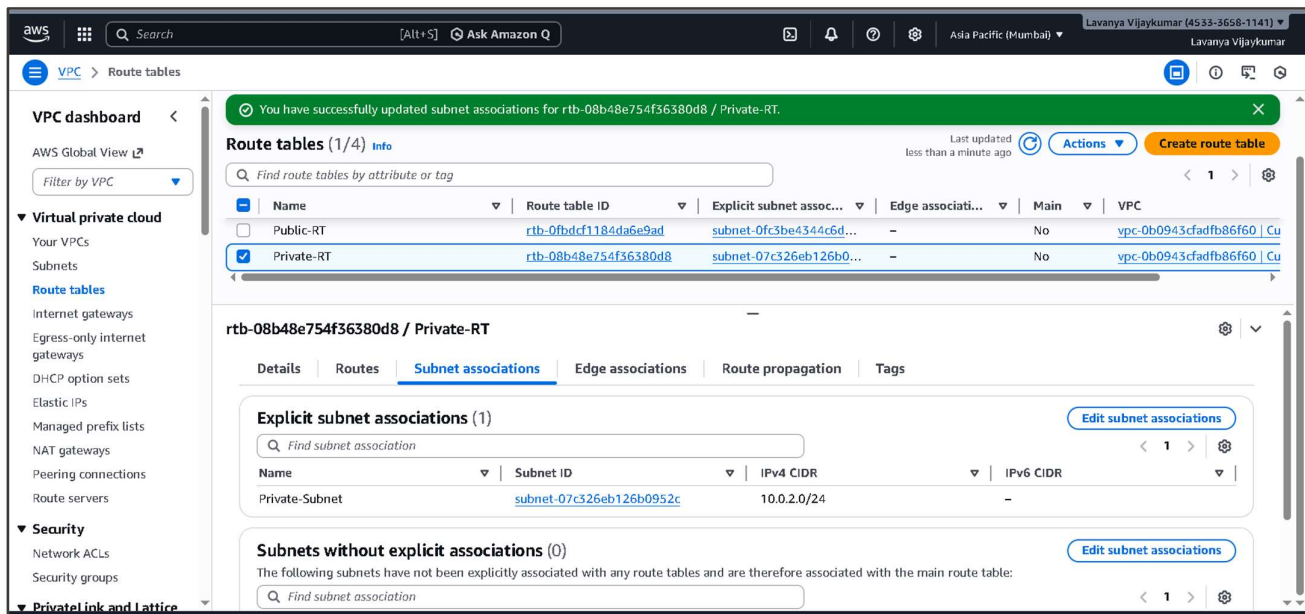
Edge associations: -

**Routes (1)**

| Destination | Target | Status | Propagated | Route Origin       |
|-------------|--------|--------|------------|--------------------|
| 10.0.0.0/16 | local  | Active | No         | Create Route Table |

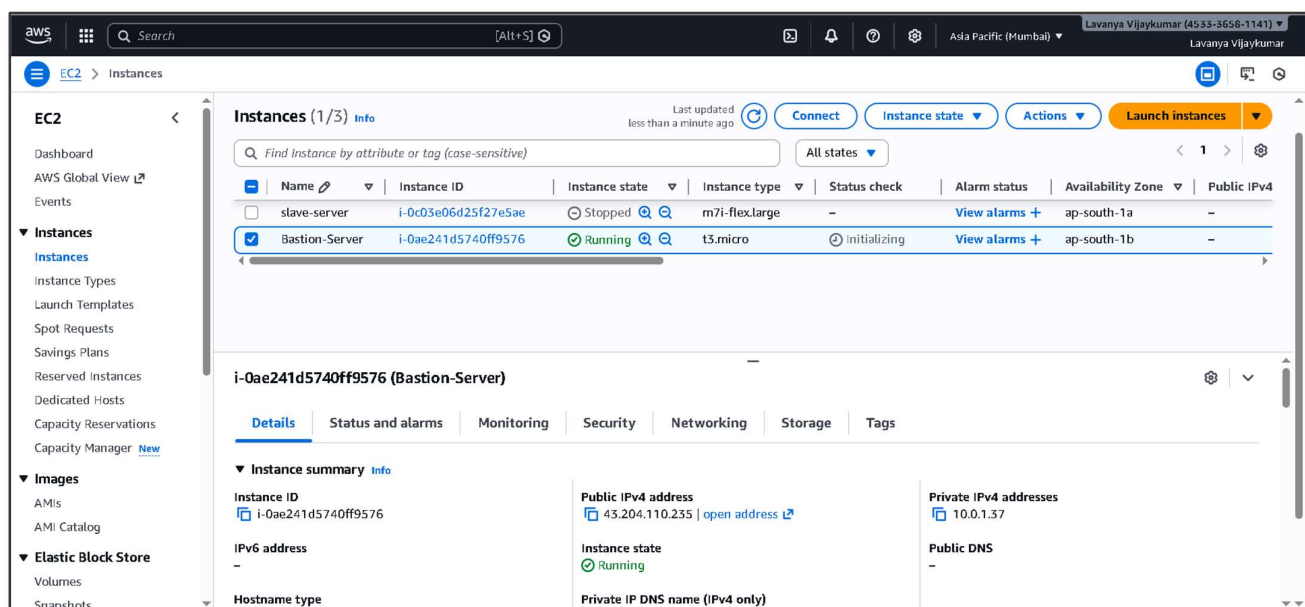
- Select the Private-RT route table from the list.
- Go to the Subnet Associations tab.
- Click Edit Subnet Associations.
- Select the Public-Subnet
- Click Save.





## Step 5: Launch Bastion-Server (Public EC2)

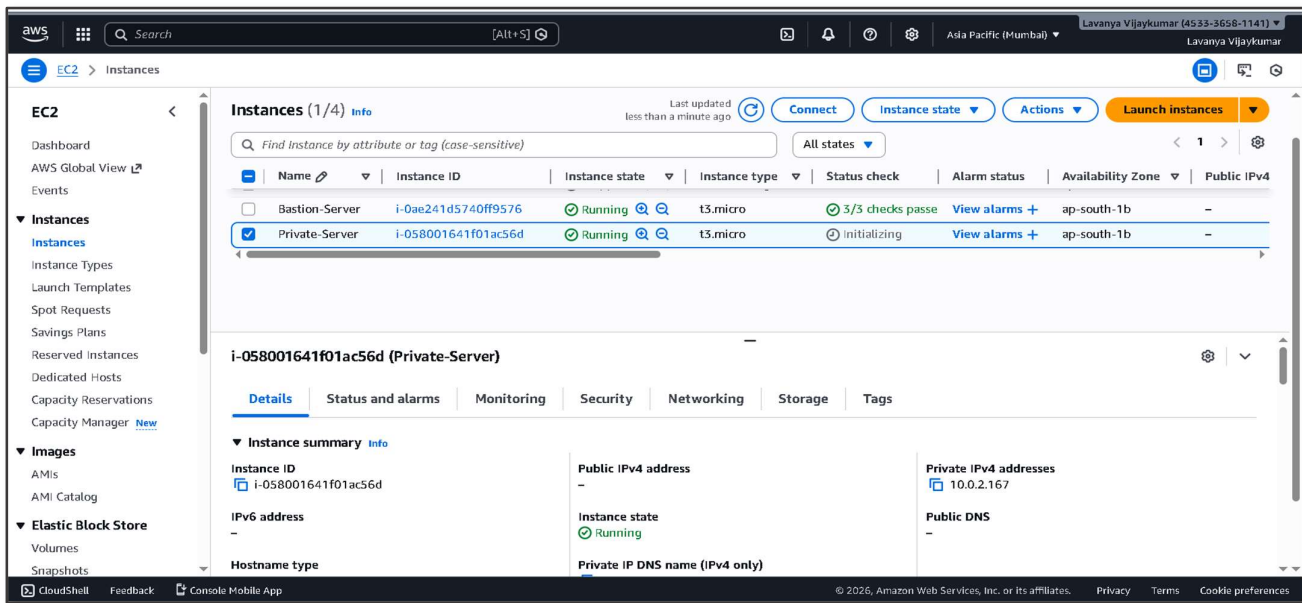
- EC2 → Launch Instance
- Name: Bastion-Server
- AMI: Amazon Linux 2
- Choose Instance Type: t3 micro.
- In Network Settings:
  1. VPC: Select your Custom VPC (Custom-VPC).
  2. Subnet: Select Public-Subnet.
  3. Auto-assign Public IP: Enable.
- Configure Security Group:
- Allow SSH (Port 22).
- Review the configuration and click Launch Instance





## Step 6: Launch Private EC2 Instance

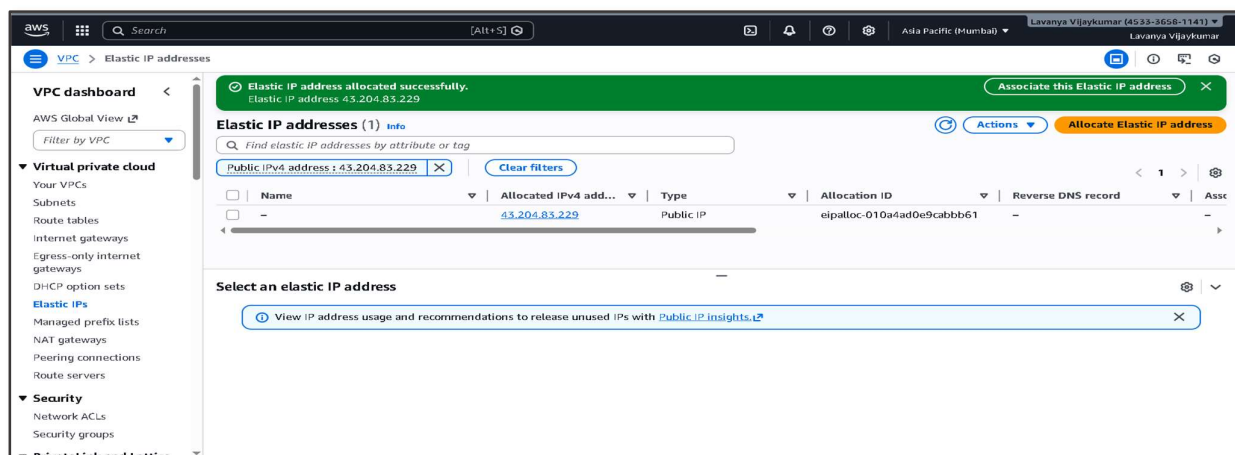
- Go to EC2 → Instances → Launch Instance.
- Enter Name: Private-EC2.
- Select AMI: Amazon Linux 2.
- Choose Instance Type: t3. micro.
- In Network Settings:
  1. VPC: Select your Custom VPC (Custom-VPC).
  2. Subnet: Select Private-Subnet.
  3. Auto-assign Public IP: Disable.
- Configure Security Group:
- Allow SSH (Port 22)
- Review the configuration and click Launch Instance.



## Step 7: Create Elastic IP and NAT Gateway

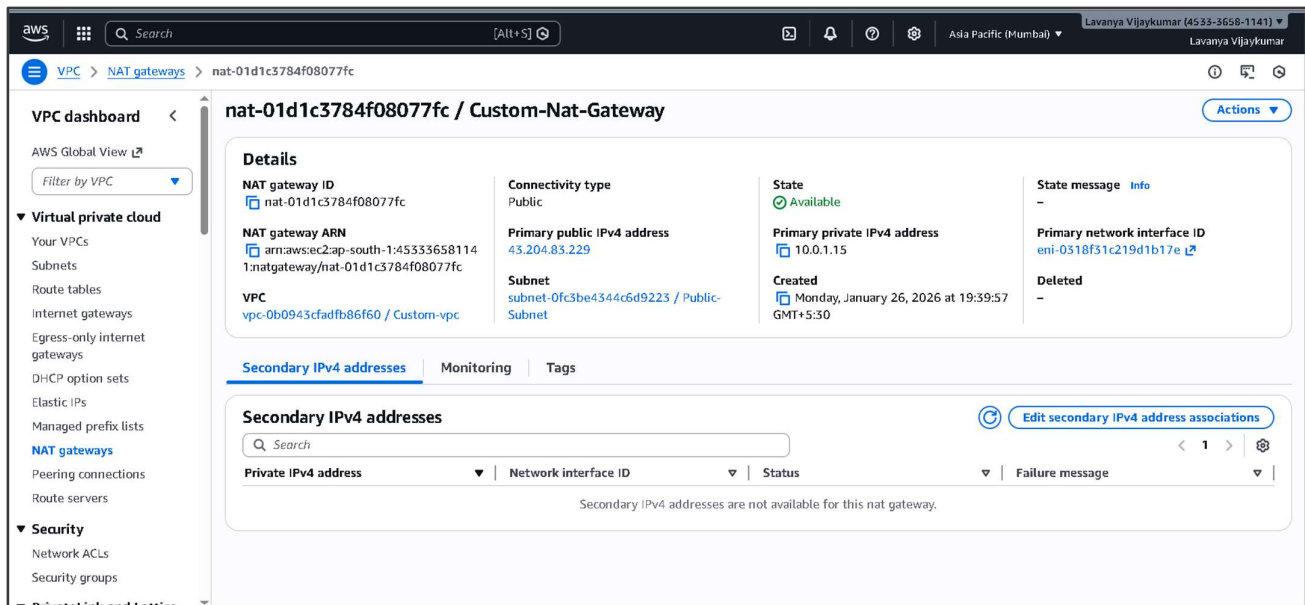
### 1.Create Elastic IP

- Go to VPC → Elastic IPs.
- Click Allocate Elastic IP address & click Allocate.



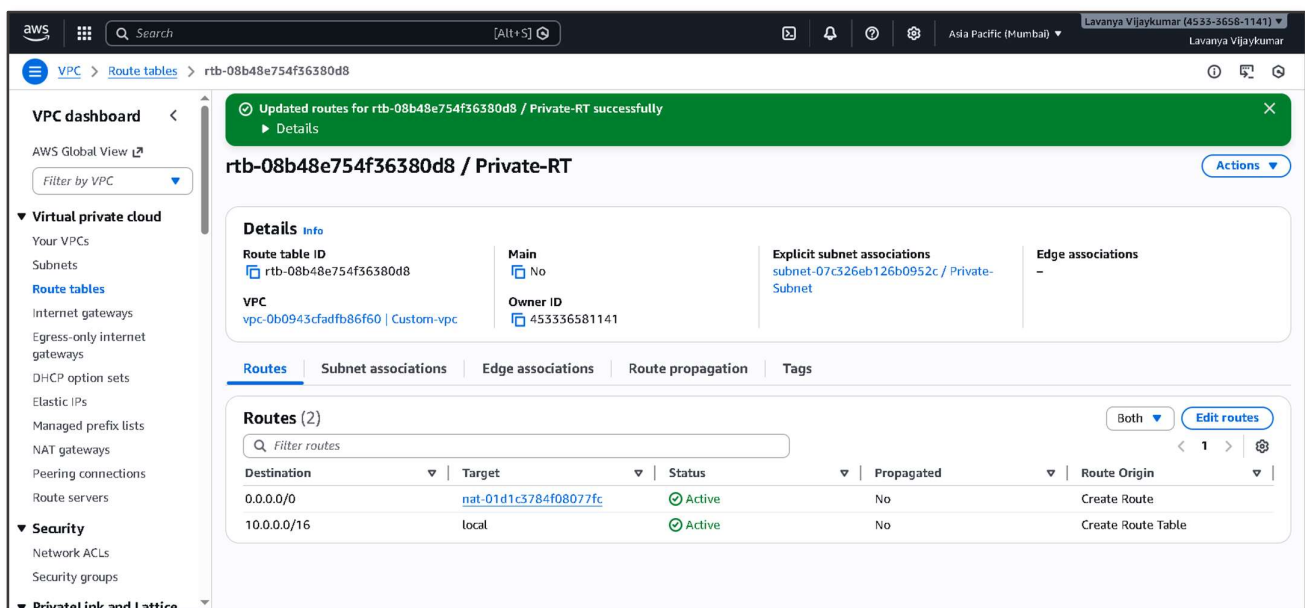
## 2.Create NAT Gateway

- Go to VPC → NAT Gateways → Create NAT Gateway.
- Enter Name: Custom-NAT-Gateway.
- Subnet: Select Public-Subnet.
- Elastic IP: Select the previously allocated Elastic IP.
- Click Create NAT Gateway & Wait until the NAT Gateway status becomes Available.



## Step 8: Update Private Route Table

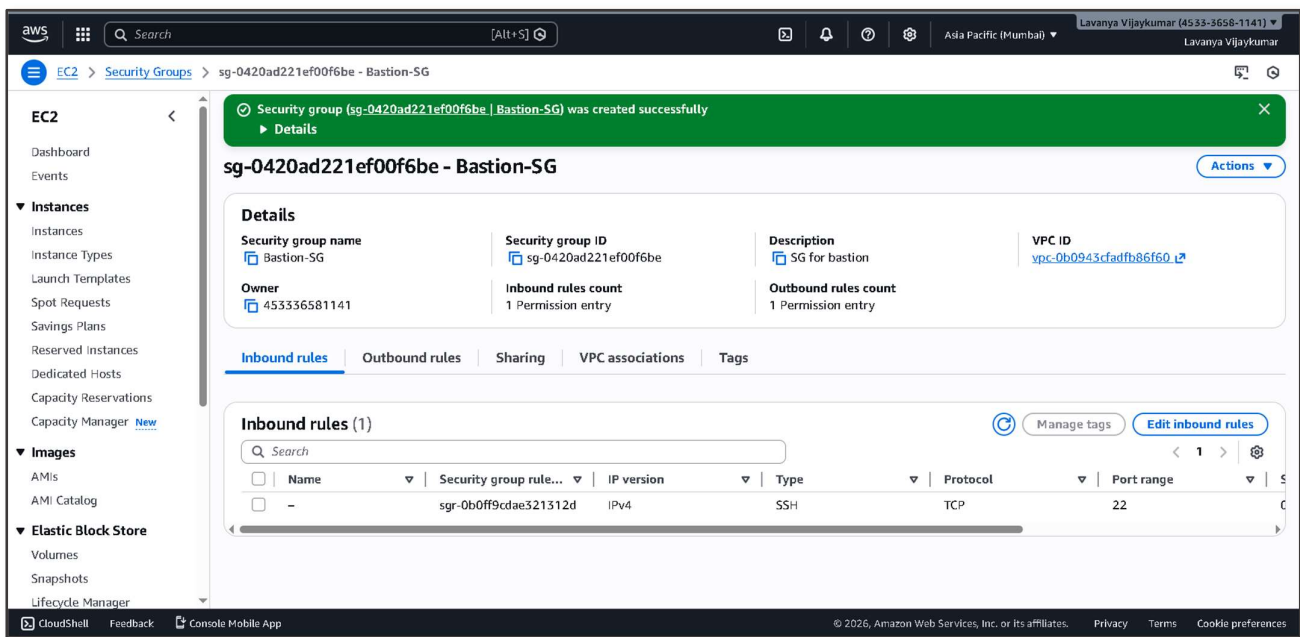
- Go to Private-RT → Edit Routes
- Add route:
  1. Destination: 0.0.0.0/0
  2. Target: NAT Gateway



## Step 9: Configure Security Groups

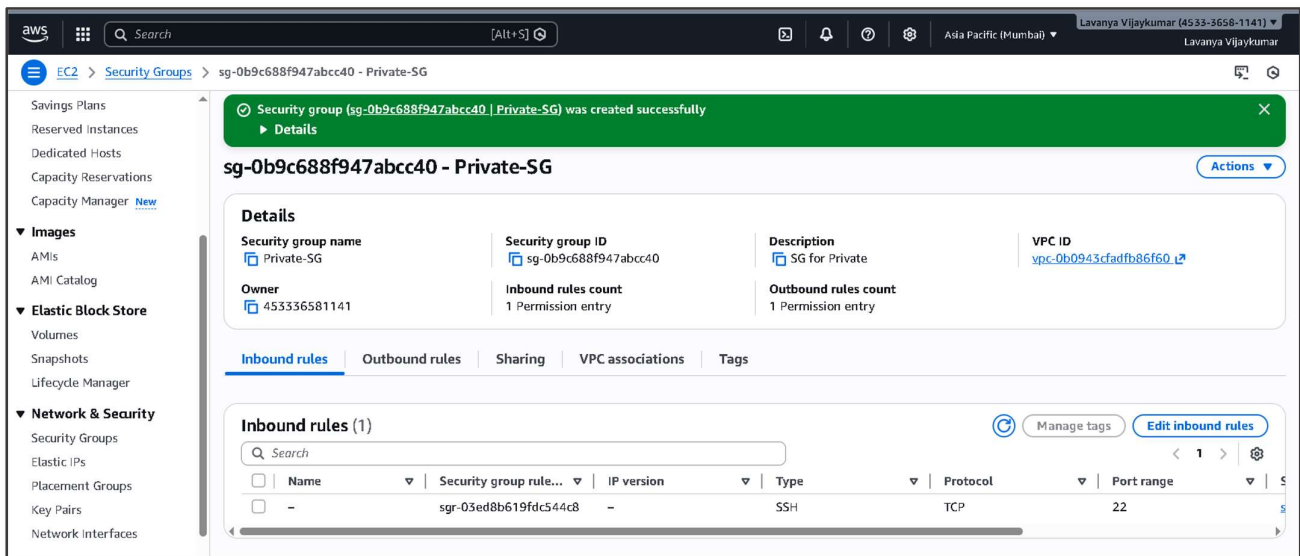
### 1. Configure Bastion Host Security Group

- Go to EC2 → Security Groups.
- Select the Bastion Host security group.
- Under Inbound rules, allow:
  1. Type: SSH
  2. Port: 22
- Source: Anywhere (0.0.0.0/0)
- Under Outbound rules, allow all traffic.
- Save the changes.



### 2. Create Private EC2 Security Group

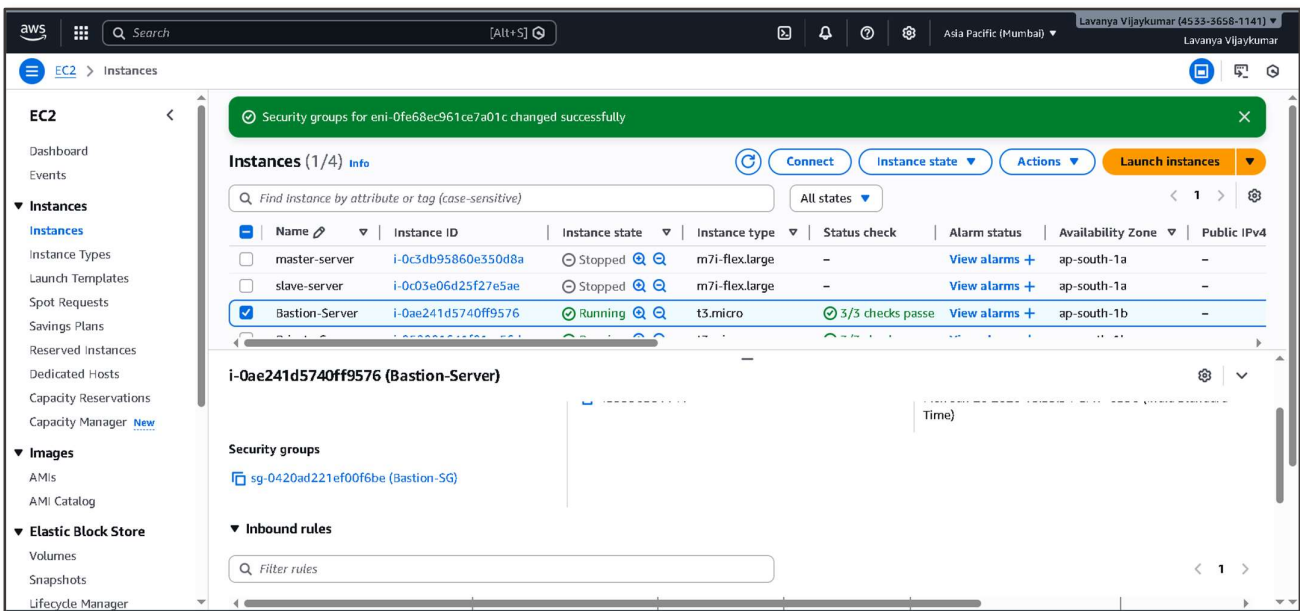
- Go to **Create security group** again.
- Enter **Security Group Name**: Private-EC2-SG.
- Select **VPC**: Custom-VPC.
- Under **Inbound rules**, add:
  1. **Type**: SSH
  2. **Port**: 22
  3. **Source**: Bastion-SG (select security group)
- Under **Outbound rules**, allow **all traffic**.
- Click **Create security group**.



## Step 10: Attach Security Groups to Existing EC2 Instances

### 1. Attach Bastion-SG to Bastion Host

- Go to EC2 → Instances.
- Select the Bastion Host EC2 instance.
- Click Actions → Security → Change security groups.
- Select Bastion-SG.
- Remove any unwanted security groups if required.
- Click Save.



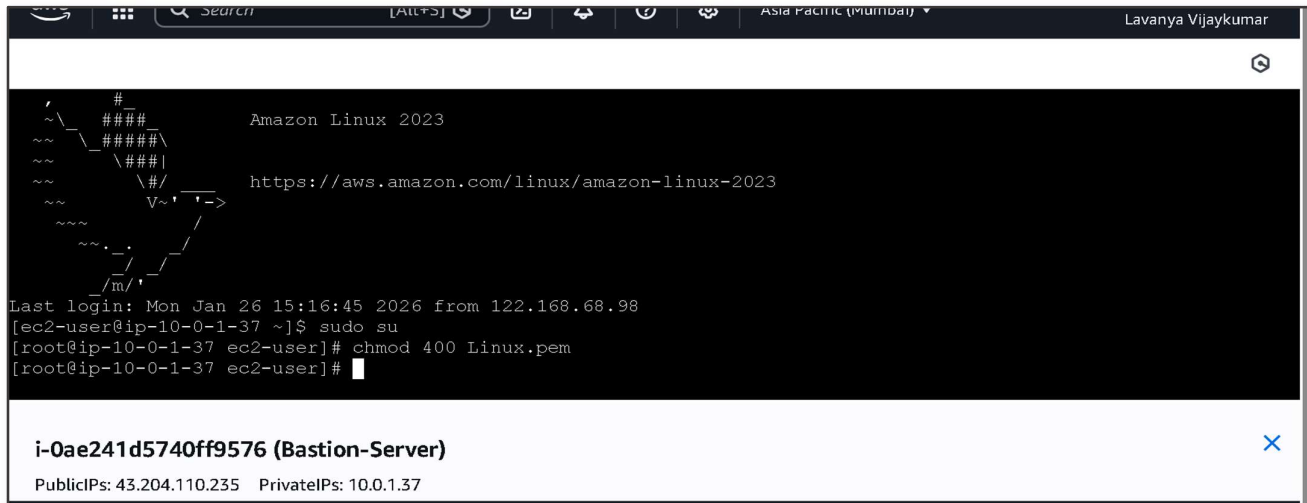
### 2. Attach Private-EC2-SG to Private EC2 Instance

- Select the Private EC2 instance.
- Click Actions → Security → Change security groups.
- Select Private-EC2-SG.
- Remove any other security groups if needed.
- Click Save.



### 3.Set Correct Permissions on the Key File (Inside Bastion)

Once logged into the Bastion Host, run: `chmod 400 key.pem`



```

~\  \#####\      Amazon Linux 2023
~\  \#####\
~\  \###|
~\  \#/
~\  V~'  -> https://aws.amazon.com/linux/amazon-linux-2023
~\  .
~\  /
~\  /m/

Last login: Mon Jan 26 15:16:45 2026 from 122.168.68.98
[ec2-user@ip-10-0-1-37 ~]$ sudo su
[root@ip-10-0-1-37 ec2-user]# chmod 400 Linux.pem
[root@ip-10-0-1-37 ec2-user]#
  
```

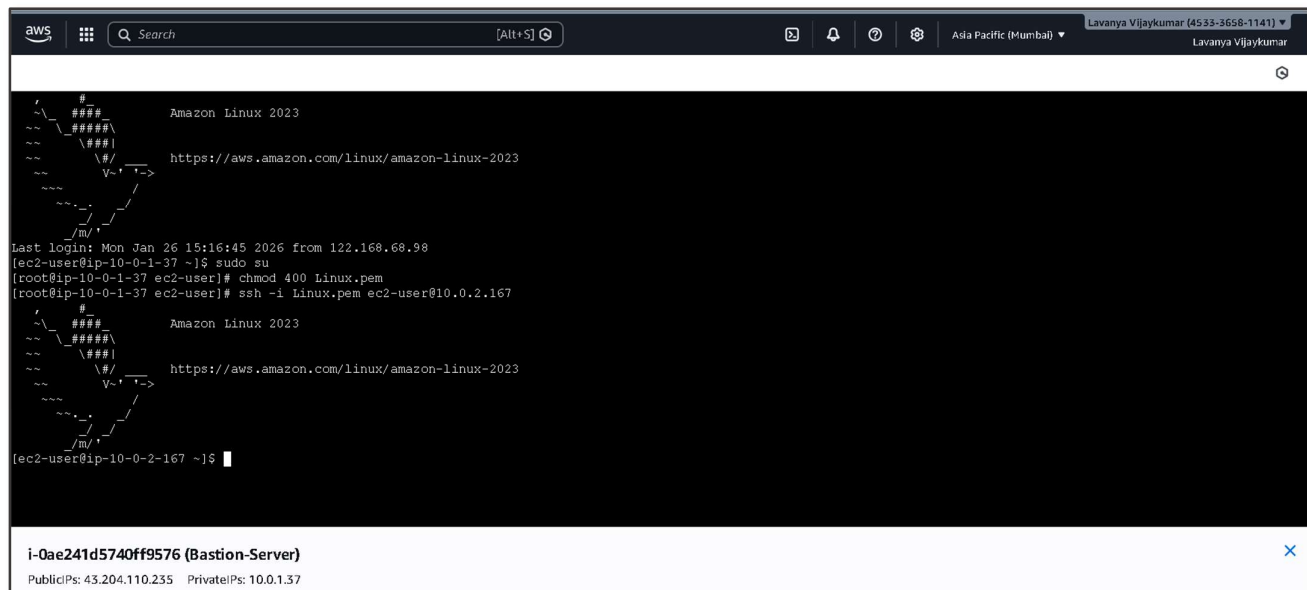
**i-0ae241d5740ff9576 (Bastion-Server)**

PublicIPs: 43.204.110.235 PrivateIPs: 10.0.1.37

### 4.Connect to Private EC2 from Bastion Host

Now run this from the Bastion Host terminal:

`ssh -i Linux.pem ec2-user@<Private_EC2_Private_IP>`



```

~\  \#####\      Amazon Linux 2023
~\  \#####\
~\  \###|
~\  \#/
~\  V~'  -> https://aws.amazon.com/linux/amazon-linux-2023
~\  .
~\  /
~\  /m/

Last login: Mon Jan 26 15:16:45 2026 from 122.168.68.98
[ec2-user@ip-10-0-1-37 ~]$ sudo su
[root@ip-10-0-1-37 ec2-user]# chmod 400 Linux.pem
[root@ip-10-0-1-37 ec2-user]# ssh -i Linux.pem ec2-user@10.0.2.167

~\  \#####\      Amazon Linux 2023
~\  \#####\
~\  \###|
~\  \#/
~\  V~'  -> https://aws.amazon.com/linux/amazon-linux-2023
~\  .
~\  /
~\  /m/

[ec2-user@ip-10-0-2-167 ~]$
  
```

**i-0ae241d5740ff9576 (Bastion-Server)**

PublicIPs: 43.204.110.235 PrivateIPs: 10.0.1.37

## Step 12: Test Internet Access from Private EC2

### 1. Update Packages

`sudo yum update -y`







## Troubleshooting:

**Issue:** While attempting to SSH into the Private EC2 instance, the connection failed with the error.

Permission denied (publickey,gssapi-keyex,gssapi-with-mic)

**Cause:** The private key file (key.pem) was not available on the Bastion Host, so SSH authentication could not succeed.

### Solution:

- Copy the key file to the Bastion Host from local machine.  
`scp -i key.pem key.pem ec2-user@<Bastion_Public_IP>:/home/ec2-user/`
- Set correct permissions on the key file inside the Bastion Host.  
`chmod 400 key.pem`
- SSH into the Private EC2 from the Bastion Host.  
`ssh -i key.pem ec2-user@<Private_EC2_Private_IP>`

After this, access to the private EC2 instance worked successfully.

## Conclusion:

In this project, a secure AWS network environment was successfully designed and implemented using a custom VPC with public and private subnets. A Bastion Host was deployed to provide controlled SSH access to private EC2 instances, ensuring network isolation and security. Additionally, a NAT Gateway was configured to allow private instances to access the internet for updates and downloads without exposing them publicly.

This project demonstrates real-world cloud networking practices that are essential for organizations. Private instances remain isolated while administrators can securely manage them via the Bastion Host, ensuring controlled access and security. The NAT Gateway provides efficient internet access for private resources without compromising security. The public/private subnet design is scalable and organized, allowing easy expansion for additional resources or applications.

By implementing this setup, organizations gain a practical understanding of AWS networking service. It equips them to build secure, efficient, and manageable cloud environments, ensuring both operational efficiency and protection of sensitive workloads.